

Music Plagiarism Detector

Inês Batista, 124877

Maria Quinteiro, 124996

Dataset MIDI (MAESTRO)

MIDI não é áudio, é uma sequência de instruções musicais, compacta e diretamente interpretável por computador. O dataset usado é o **MAESTRO (Google)**, com gravações reais de pianistas em competições internacionais.

Estrutura de um ficheiro MIDI



Cabeçalho MThd

Guarda o formato, número de faixas e *ticks_per_quarter*.



Faixas MTrk

Sequência de eventos separados por *delta-time*. Eventos como *note on* e *off*.



Conversão de Ticks

Divide pelo *ticks_per_quarter* para obter uma representação invariante ao andamento.

Tratamento de dados MIDI

1. Leitura e Emparelhamento

Leitura em big-endian. Emparelha note on/off por canal e nota para calcular onset e duração.

2. Filtragem de Eventos

Eventos irrelevantes (control change, pitch bend, meta) são ignorados para focar na melodia.

3. Ordenação Final

Ordenação por onset. Em caso de empate temporal, a nota mais aguda (maior número MIDI) surge primeiro na sequência.

```

ticks_per_quarter = double(division);
events = empty_events();

% Parse every track chunk and collect note events from all tracks.
for track_idx = 1:num_tracks
    if feof(fid)
        break;
    end

    track_type = read_chunk_type(fid);
    track_len = fread(fid, 1, 'uint32');
    track_data = fread(fid, track_len, 'uint8');

    if strcmp(track_type, 'MTrk')
        track_events = parse_track(track_data, ticks_per_quarter, track_idx);
        events = [events, track_events]; %#ok<AGROW>
    end
end

% Keep events in musical order before returning them.
events = sort_events(events);

```

```

event.onset_beat = onset_tick / ticks_per_quarter;
event.duration_beat = duration_tick / ticks_per_quarter;
event.velocity = on_velocity;
event.off_velocity = double(off_velocity);
event.channel = channel;
event.track = track_idx;
events(end + 1) = event;

```

Sequência (intervalo, duração)

1. top_note_per_onset

Resolve polifonias selecionando a nota mais aguda por onset. Em empates, a nota de maior duração prevalece.

2. Cálculo de Intervalos

Diferença entre números MIDI consecutivos, preservando o sentido (+ ascendente, - descendente).

3. Matriz Nx2 Resultante

Par [intervalo melódico, duração quantizada]. N notas geram N-1 pares (a primeira nota não gera intervalo).

```
idx] = sortrows([-notes(:), -durations(:)], [1, 2]);  
ected = events(same_onset(idx(1)));
```

```
% Split the melody into pitch and duration vectors. Durations are  
% quantized first so small performance differences do not dominate matching  
notes = [melody.note_number];  
durations = [melody.duration_beat];  
quantized_durations = quantize_values(durations, duration_grid);  
  
% Intervals represent melodic movement between consecutive notes.  
% Signed intervals preserve direction; mod12 can be enabled if only pitch  
% class movement matters.  
intervals = diff(notes);  
if interval_mod12  
    intervals = mod(intervals, 12);  
end  
  
% Pair each interval with the duration of the note it arrives on. The first  
% note has no previous note, so its duration is not used in the sequence.  
sequence = [intervals(:), quantized_durations(2:end)'];
```

K-Shingles

É uma janela deslizando de k pares consecutivos da sequência. **Com $k=8$ e N pares produzem-se $N-7$ janelas sobrepostas.** Dois fragmentos que diferem apenas numa nota ainda partilham vários shingles, tornando a similaridade proporcional ao grau real de sobreposição melódica.

Cada janela é serializada intercalando intervalos e durações sendo que as durações são multiplicadas por 1000 antes do hash para evitar problemas de vírgula flutuante. O hash polinomial usa base 131 (primo acima de 128), constante 1000003 (afasta valores de zero) e módulo $2^{31}-1$ (primo de Mersenne). Duas janelas idênticas produzem sempre o mesmo hash. **O conjunto final é de hashes únicos já que os duplicados colapsam.**

Bloom Filter

Array de bits inicialmente a zero com múltiplas posições calculadas por duplo hashing - duas funções base (h_1 e h_2) são combinadas linearmente para simular k funções independentes sem as calcular todas.

- Inserir um shingle: coloca os bits nas posições calculadas a 1
- Verificar um shingle: testa se todos os bits estão a 1
- Se algum bit estiver a 0 → definitivamente não está (**sem falsos negativos**)
- Se todos a 1 → provavelmente está (**falsos positivos possíveis**)

No projeto, `bloom_count_matches` conta quantos shingles da consulta passam no filtro de uma melodia da base de dados. **Só melodias com rácio acima de 0.5 avançam para o cálculo exato de Jaccard.**

```
function result = bloom_check(bf, shingle_hash)
    % Checks if a shingle might be in the filter.
    % Returns true if it MIGHT be there (could be a false positive).
    % Returns false if it is DEFINITELY not there (never a false negative).
    positions = get_bit_positions(shingle_hash, bf.num_hashes, bf.num_bits);
    result = all(bf.bits(positions));
end
```

```
function positions = get_bit_positions(hash_val, num_hashes, num_bits)
    % Derives num_hashes bit positions from a single hash value.
    % Uses double hashing: simulates multiple hash functions from just two.
    h1 = mod(hash_val, num_bits) + 1;
    h2 = mod(floor(hash_val / num_bits), num_bits) + 1;
    positions = zeros(1, num_hashes);
    for i = 1:num_hashes
        positions(i) = mod(h1 + (i - 1) * h2, num_bits) + 1;
    end
    positions = unique(positions);
end
```

MinHash + LSH

MinHash: Assinaturas

Comprime shingles em 100 inteiros. **Para cada posição, guarda-se o hash mínimo.**

Probabilidade de coincidência = Similaridade de Jaccard.

LSH: Filtragem

Divide a assinatura em **20 bandas de 5 posições**. Reduz a complexidade de $O(n^2)$ para apenas pares candidatos.

Cálculo Final (Candidatos)

$$J(A, B) = |A \cap B| / |A \cup B|$$

Semente rng(42) garante reprodutibilidade total das funções de hash.

```

rng(42); % Para reprodutibilidade
a = randi([1, max_shingle], 1, num_hashes);
b = randi([0, max_shingle], 1, num_hashes);
p = 4294967311; % primo grande
sig = zeros(1, num_hashes);
for i = 1:num_hashes
    hashes = mod(a(i) * double(shingle_set) + b(i), p);
    sig(i) = min(hashes);
end

```

```

% Calcula similaridade Jaccard real para os candidatos
keys = candidate_pairs.keys;
similar_pairs = [];
for k = 1:length(keys)
    pair = candidate_pairs(keys{k});
    set1 = shingle_sets(pair(1));
    set2 = shingle_sets(pair(2));
    sim = jaccard_similarity(set1, set2);
    if sim >= threshold
        similar_pairs = [similar_pairs; pair];    Variable appears to change size
    end
end

```

Classificador Naïve Bayes

Conceito Principal

Usado para classificar melodias como "**plagiadas**" ou "**originais**". Utiliza a Implementação Multinomial para contagens de shingles, baseando-se na independência de features do Teorema de Bayes.



Logaritmos

Evita underflow numérico ao lidar com produtos de probabilidades extremamente pequenas.



Laplace Smoothing

($\alpha=1$) Garante que shingles novos não anulem a probabilidade final da classe durante a inferência.



Dados de Treino

Suporta matrizes ou cell arrays. O vocabulário é extraído automaticamente dos dados de treino.

Testes e Conclusão